

Atividade JWT e swagger

<http://localhost:8080/swagger-ui/index.html#/>

authentication-controller

POST /auth/register Register

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "login": "Ze",
  "password": "12345",
  "role": "ADMIN"
}
```

Request URL

http://localhost:8080/auth/register

Server response

Code Details

200

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 0
date: Fri, 09 Aug 2024 23:18:32 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Ao realizar Login
Iniciamos registrando o
usuário no sistema.
Observe que ele possui a
função de ADMIN.

PUT	/pessoas	✓	🔒
POST	/pessoas	✓	🔒
GET	/pessoas/pessoa	✓	🔒
GET	/pessoas/pessoa/{id}	✓	🔒
DELETE	/pessoas/pessoa/{id}	✓	🔒
authentication-controller		^	
POST	/auth/register Register	✓	🔒
POST	/auth/login Login	✓	🔒

POST

/auth/login

Login

^

🔒

Parameters

Try it out

No parameters

Request body

required

application/json

▼

```
{
  "login": "ze",
  "password": "12345"
}
```

Execute

Clear

Server response

Code

Details

200

Response body

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJhdXRoLWZwaSI6InN1YiI6IiIiwiaXNjaXNzIjMjUyODkyfQ.CZUxq5ebCZU7x75Xg1UsAKfGN6aZ1VarN1y4aHb3Pic"
}
```

📄

Download

Ao realizar Login com um usuário presente na base de dados é retornado um token que vai auxiliar para o acesso das outras requisições

Minha API 1.0 OAS3

/v3/api-docs

Servers

http://localhost:8080 - Generated server url

Authorize



peessoa-controller

PUT

/pessoas

Minha API 1.0 OAS3

/v3/api-docs

Servers

http://localhost:8080 - Generat

peessoa-controller

PUT

/pessoas

POST

/pessoas

Available authorizations

x

bearer-key (http, Bearer)

Value:

```
{ "token": "eyJhbGciOiJIUzI1
```

Authorize

Close

Informar o token obtido na etapa anterior.

GET

/pessoas

^

🔒

Parameters

Cancel

No parameters

Execute

Clear

Responses

Consulta de todas as pessoas.

Server response	
Code	Details
200	Response body <pre>[{ "id": 1, "nome": "Zé mané" }]</pre> 📄 Download Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive</pre>

POST /pessoas

Parameters

No parameters

Request body required

application/json

```
{
  "nome": "Maria"
}
```

Cadastrando uma nova pessoa.

Server response

Code	Details
200	Response body <pre>{ "id": 2, "nome": "Maria" }</pre> Download

Ao colocar o token e tentar acessar a requisição de registro, agora funciona e o novo usuário é salvo na tabela, importante destacar que isso só foi possível pois **zé** foi quem realizou o cadastro ele é um usuário do tipo ADMIN.

GET /pessoas

Parameters

No parameters

Execute Clear

Responses

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "nome": "Zé mané" }, { "id": 2, "nome": "Maria" }]</pre> Download

Consulta de todas as pessoas.

O método de listagem de pessoas pode ser acessado por todos os tipos de usuário, bastando apenas estar logado

GET

/pessoas/{id}

^

🔒

Parameters

Cancel

Name	Description
id * required	
integer(\$int64)	1
(path)	

Execute

Clear

Responses

Consultar uma pessoa.

Code

Details

200

Response body

```
{
  "id": 1,
  "nome": "Zé mané"
}
```

📄

Download

PUT

/pessoas/{id}

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int64) (path)	1

Request body required

application/json

```
{  
  "nome": "Zé Mané"  
}
```

Alterar uma pessoa.

Code

Details

200

Response body

```
{  
  "id": 1,  
  "nome": "Zé Mané"  
}
```

Download

DELETE

/pessoas/{id}

^

🔒

Parameters

Cancel

Name	Description
id * required	
integer(\$int64)	1
(path)	

Execute

Exclui uma pessoa.

GET

/pessoas

^

🔒

Parameters

Cancel

No parameters

Execute

Clear

Responses

Code

Details

200

Response body

[
 {
 "id": 2,
 "nome": "Maria"
 }
]

📄

Download

Post Admin

SQL File 3* aluno tb_users x

Limit to 1000 rows

1 • `SELECT * FROM mydb.tb_users;`

Result Grid

	id	login	password	role
▶	1	yan	\$2a\$10\$86l.HB2XLBMh2mJ7B1heFu0onF6SvG5t...	0
	2	admin	\$2a\$10\$00LIL/UH1oXXXW4x8EwbRONfs0yaZv...	0
*		NULL	NULL	NULL

Request body required

```
{
  "login": "admin",
  "password": "123",
  "role": "ADMIN"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/auth/register' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJhdXRoLWwSIsInN1YiI6InlhbGlIsImV...' \
  -H 'Content-Type: application/json' \
  -d '{
    "login": "admin",
    "password": "123",
    "role": "ADMIN"
  }'
```

Request URL

`http://localhost:8080/auth/register`

Server response

Code	Details
200	Response headers

Login

Request body

```
{
  "login": "admin",
  "password": "123"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://localhost:8080/auth/login' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJhdXRoLFwSIsInN1YiI6InlubiIsImV4cCI6MTcyMzA4NjIyOH0.eyJ0b3R82B83QhQd040U6o560z6j3ADgYDibnEjqR_g' \
-H 'Content-Type: application/json' \
-d '{
  "login": "admin",
  "password": "123"
}'
```

Request URL

```
http://localhost:8080/auth/login
```

Server response

Code	Details
200	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJhdXRoLFwSIsInN1YiI6ImFkbWluIiwiaXNjaXoxNzIzMDg5MzgufQ.eyJ0b3R82B83QhQd040U6o560z6j3ADgYDibnEjqR_g" }</pre>

Post pessoa (necessário token)

Request body required

```
{
  "nome": "Junior"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/pessoas' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJhdXRoLWFWaSI6InN1YiI6ImFkbWluIiwiaXNjaXNzIzMDg5MzgwZjQubVe3k7vPm1VF8xQLwDPEno0JE2-eqaX6HQ0xHcIump0' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "Junior"
  }'
```

Request URL

http://localhost:8080/pessoas

Server response

Code	Details
200	Response body <pre>{ "id": 2, "nome": "Junior" }</pre>

Get pessoa (não é necessário token)

The screenshot shows a REST client interface with the following sections:

- GET /pessoas/pessoa**: The method and endpoint.
- Parameters**: A section with a "Cancel" button and the text "No parameters".
- Execute**: A blue button to execute the request.
- Clear**: A button to clear the request.
- Responses**: A section containing:
 - Curl**: A text box with the command: `curl -X 'GET' \ 'http://localhost:8080/pessoas/pessoa' \ -H 'accept: */*'`
 - Request URL**: A text box with the URL: `http://localhost:8080/pessoas/pessoa`
 - Server response**: A section with tabs for **Code** and **Details**.
 - Code**: Shows the status code **200**.
 - Response body**: A text box containing a JSON array:

```
[
  {
    "id": 1,
    "nome": "string"
  },
  {
    "id": 2,
    "nome": "Junior"
  }
]
```

 with a "Download" button.
 - Response headers**: A text box containing the following headers:

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Thu,08 Aug 2024 02:01:15 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
```